

Dual-Sparse Compressed Systolic Acceleration for SNN workloads

Introduction and Motivation

Spiking Neural Networks (SNNs) offer a compelling energy profile relative to Artificial Neural Networks (ANNs): computation is event driven, and the core operation reduces to accumulate (AC) rather than a full multiply-accumulate (MAC). Despite this advantage, SNNs have not displaced ANNs, principally because most existing SNN accelerators process timesteps sequentially, for a network evaluated over T timesteps, inference latency scales linearly with T even though each timestep's computation is individually inexpensive.

Two observations motivate this work. First, sequential timestep processing is a dataflow artifact rather than an intrinsic property of the presynaptic accumulation step. The dataflow can be restructured to accumulate all T timesteps contributions in a single parallel step, eliminating timestep count as a latency factor in that phase.

Second, SNNs exhibit substantially higher sparsity than ANNs, weight pruning and spike sparsity together exceed 90%. Under a dense systolic mapping, PE cycles are spent on zero-valued operands, and the full dense matrix must be stored and streamed regardless of how few entries are nonzero. Offline sparse packing addresses this on three fronts at once. It reduces on-chip storage, since a packed matrix

requires only its compressed dimensions plus a small index-tag overhead rather than the full dense footprint. It reduces array traversal latency, since completion time scales with the packed operand's dimensions. And it improves effective PE utilization as a consequence of the first two. These three effects: memory, speedup, and utilization, are the return on doing packing offline, before any data reaches the array.

Architecture

2.1 Parallel Timestep Accumulation

The PE performs all T gated accumulations in one cycle instead of T separate passes. Each bit of the incoming spike mask controls only its own timestep's accumulator, bit ' t ' decides whether the stationary weight gets added to timestep t 's running sum. Since summing across timesteps has no order dependency, there's no reason to do this one timestep at a time. A naive accelerator that processes $T=8$ timesteps sequentially re-traverses the whole array 8 times before it even gets to LIF. Doing all 8 at once removes T as a cost here entirely, this stage now only depends on operand size, not on how many timesteps the network runs for.

The LIF membrane update depends on order: a spike at timestep t changes what timestep $t+1$ accumulates against, via the reset. The sequence itself is unrolled combinatorially into a single cycle rather than run over T actual clock cycles. It runs after all T accumulations for a given output are already

done and it runs as soon as that particular output is ready rather than waiting on the rest of the array.

2.2 Dataflow Selection

We use weight-stationary (WS) instead of output-stationary (OS) or input-stationary (IS). Under WS, a weight loads once and stays put while every activation it needs to multiply against streams past it. This lines up with how neural nets actually behave: one weight gets reused across many activations, but one activation typically only pairs with a given weight once. So WS keeps the operand that's reused a lot (the weight) stationary, and streams the operand that isn't (the activation). IS does the opposite, keeps activation stationary, streams the weight, which reuses the wrong operand for this kind of workload.

OS makes a different tradeoff: instead of keeping an operand still, it keeps the partial sum still and streams both operands through. That avoids writing partial sums back and forth, but since neither weight nor activation stays resident, both have to be re-fetched constantly for every PE that needs them, more total data movement than WS, even though OS saves on moving partial sums specifically.

2.3 Offline Sparse Packing

Packing compresses weights by row and activations by column, entries that don't collide (no overlapping nonzero positions) get merged into the same physical slot. This all happens offline, before the PE array ever sees the data, so

the compression cost is paid once up front, not repeatedly during execution. Every merged entry keeps a small index tag where it originally came from.

Without packing, each PE maps to exactly one output position, so one accumulator per PE is enough. Packing breaks that, once several original columns get merged into one physical column of B^* , a single PE might now be asked to compute several different outputs over time, not just one, depending on which merged column the current data actually belongs to. To tell these outputs apart and route each contribution to the right one, the PE needs the tags and dimensions carried alongside the packed data, and it needs more than one accumulator to hold them simultaneously.

So instead of one accumulator, each PE gets a small buffer of a few slots, each one tagged with which output it belongs to. When new data arrives, its tag gets checked against the slots already in use: matching tag adds into that slot; no match grabs a free slot and starts a new one. That's what lets one PE juggle multiple unrelated outputs at once without them getting mixed up, the tag tells you which output a value belongs to, not just where in the array it happens to be sitting.

2.4 Latency and LIF Control Signals

Each partial sum moves left to right across the array, one column of PEs at a time, until it exits the fourth and final column. The moment a C^* value exits that last column, the

very next cycle the unrolled, single-cycle LIF computation is performed on it and the result is handed to the post-processing unit. This happens continuously as results exit, not as one batch at the end, each C^* value gets its LIF pass the cycle immediately after it clears the array.

Given a packed weight of dimensions $M \times K$ and a packed activation of dimensions $K \times N$, streamed into an array of fixed width COLS, the latency for the last result to exit is:

$$\text{Latency} = (N - 1) + M + (\text{COLS} - K)$$

N accounts for the last activation column entering the array, m accounts for it traveling down to the last weight-bearing row, and $(\text{COLS} - K)$ accounts for the result then crossing the remaining empty columns to reach the physical edge.

This is fixed once the dimensions are known and does not depend on how the operand's sparsity is arranged internally. A counter tracks this countdown in the background while results continue exiting and getting their LIF pass as described above. When the counter expires, the very last C^* has just exited and received its LIF pass, and that event is the trigger for a control signal telling the post-processing unit that this matrix is fully done, at which point it can begin processing the completed output while the pre-processing unit fetches and packs the next matrix for the array.